AWS APPLICATION LOAD BALANCER

High Availability & Health Check Project

Hands-on Project Documentation

Region: us-east-2 (Ohio) | Load Balancer: ALB-Project | Target Group: ALB-Target-Group

1. Project Overview

This project demonstrates how an internet-facing AWS Application Load Balancer (ALB) distributes HTTP traffic across two Amazon EC2 web servers. Both instances run Amazon Linux 2023 and Apache HTTP Server. Each server is configured with a different welcome page so that the backend response can be distinguished.

A failure scenario was tested by stopping Apache on Server 1. The target group health check marked that target unhealthy while Server 2 remained healthy. Apache was then started again on Server 1, and the target returned to a healthy state.

2. Project Objectives

- Create two EC2 web servers using Amazon Linux 2023.
- Install and run Apache HTTP Server on both instances.
- Register both instances in an EC2 target group.
- Create an internet-facing Application Load Balancer with an HTTP listener on port 80.
- Verify target health checks and normal operation.
- Simulate an application-server failure and observe the unhealthy target state.
- Restore the server and verify that it becomes healthy again.

3. Architecture

Internet
↓
Application Load Balancer (HTTP : 80)
↓
ALB-Target-Group
↙️ ↘️
ALB-Server-1 ALB-Server-2
Apache : 80 Apache : 80

4. AWS Components Used

Component	Purpose
Amazon EC2	Runs the two backend web servers.
Application Load Balancer	Receives HTTP requests and forwards them to registered targets.
Target Group	Registers EC2 instances and performs health checks.
Listener	Accepts HTTP traffic on port 80 and forwards requests to the target group.

VPC and public subnets	Provide the network environment for the ALB and EC2 instances.
Security Groups	Control inbound and outbound network access.
Internet Gateway	Provides internet connectivity for the internet-facing ALB setup.
Amazon Linux 2023 + Apache	Hosts the sample web pages on both EC2 instances.

5. Implementation Steps and Evidence

Step 1 – Create EC2 Web Servers

Two EC2 instances were created for the project and named ALB-Server-1 and ALB-Server-2. The supplied console screenshot shows both instances in the Running state and their EC2 status checks passed.

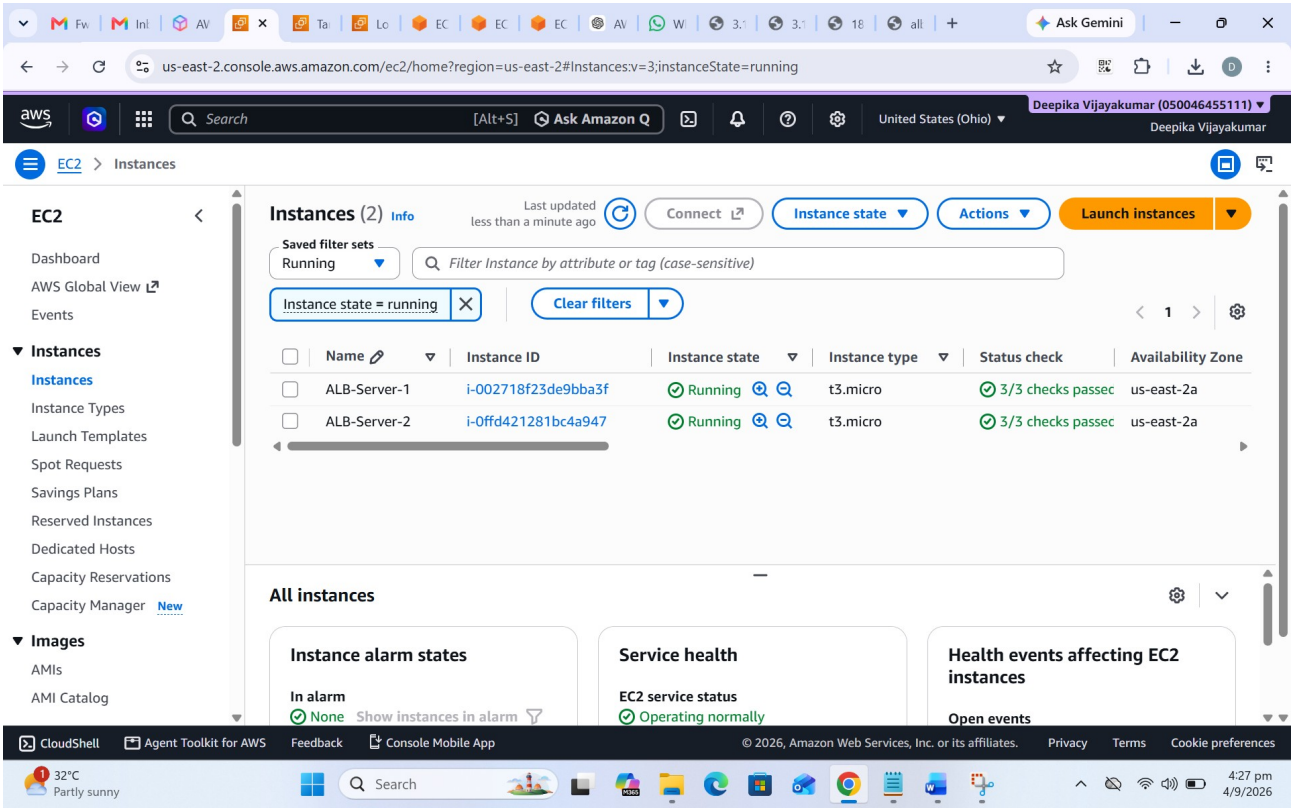


Figure 1 – EC2 instances: ALB-Server-1 and ALB-Server-2 running.

Step 2 – Install and Start Apache on Both Servers

Apache HTTP Server was installed, started, and enabled on both Amazon Linux 2023 instances. The service status output confirms that httpd was active (running) and listening on port 80.

Commands used:

```
sudo dnf update -y
sudo dnf install -y httpd
sudo systemctl start httpd
sudo systemctl enable httpd
sudo systemctl status httpd
```

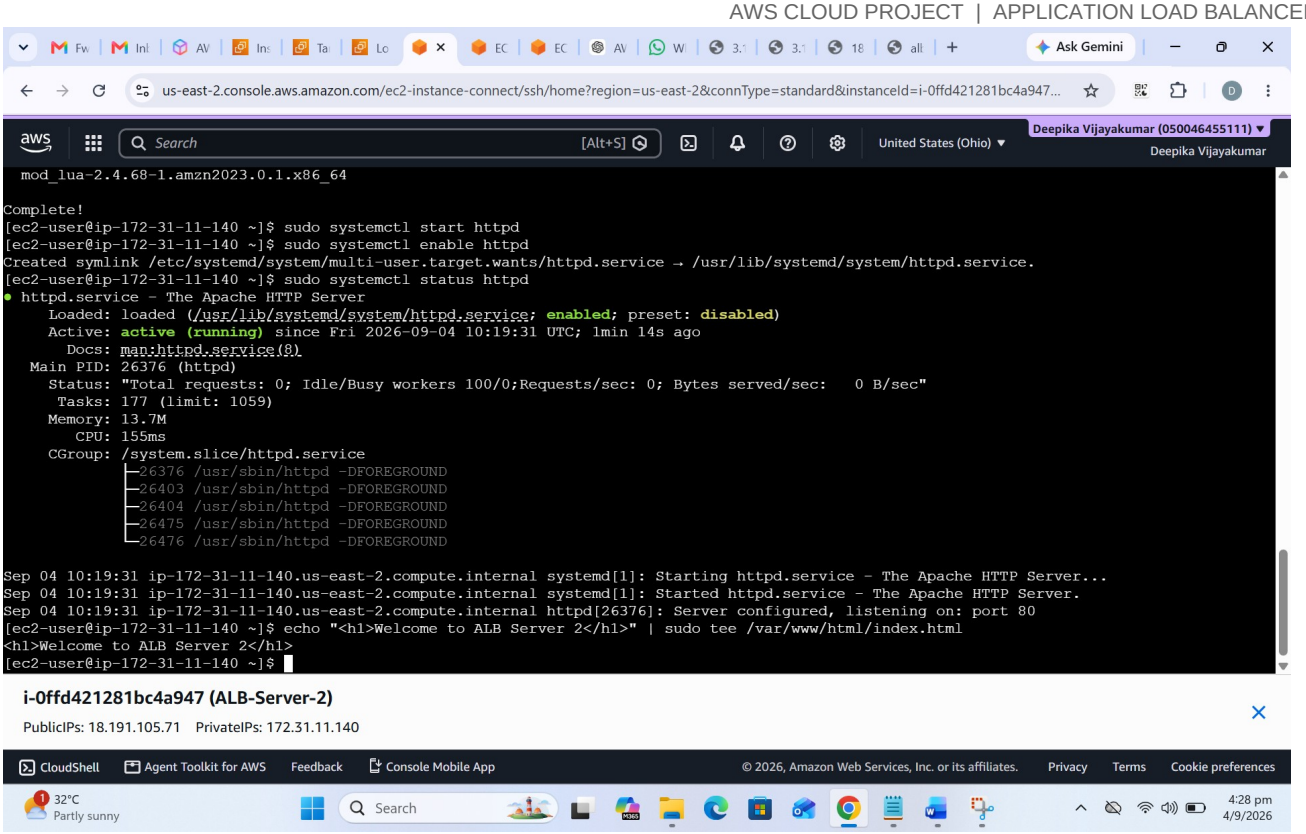


Figure 2 – Apache HTTP Server running on ALB-Server-1.

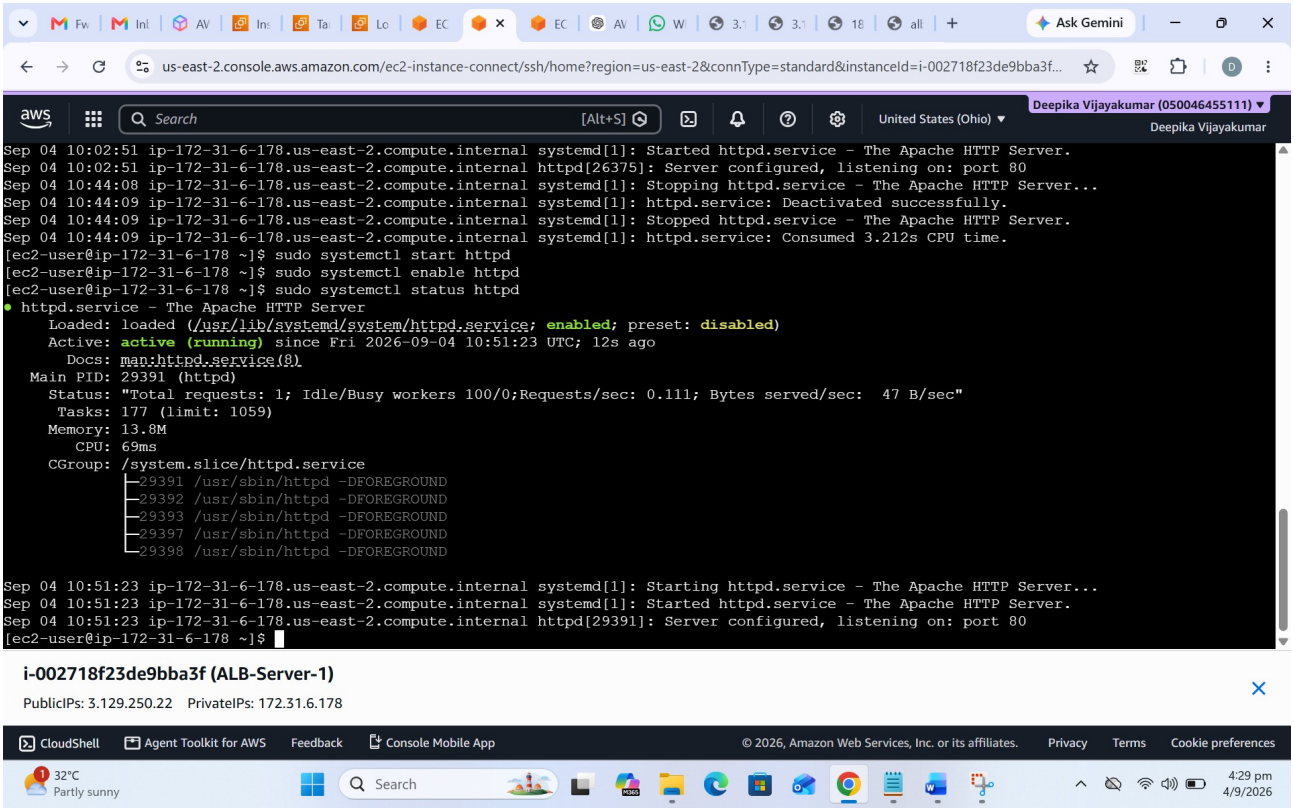


Figure 3 – Apache HTTP Server running on ALB-Server-2.

Step 3 – Create the Target Group

A target group named ALB-Target-Group was created with instance targets, HTTP protocol, port 80, and an HTTP health check using the root path (/).

- Target type: Instances
- Protocol and port: HTTP : 80
- Health check protocol: HTTP
- Health check path: /

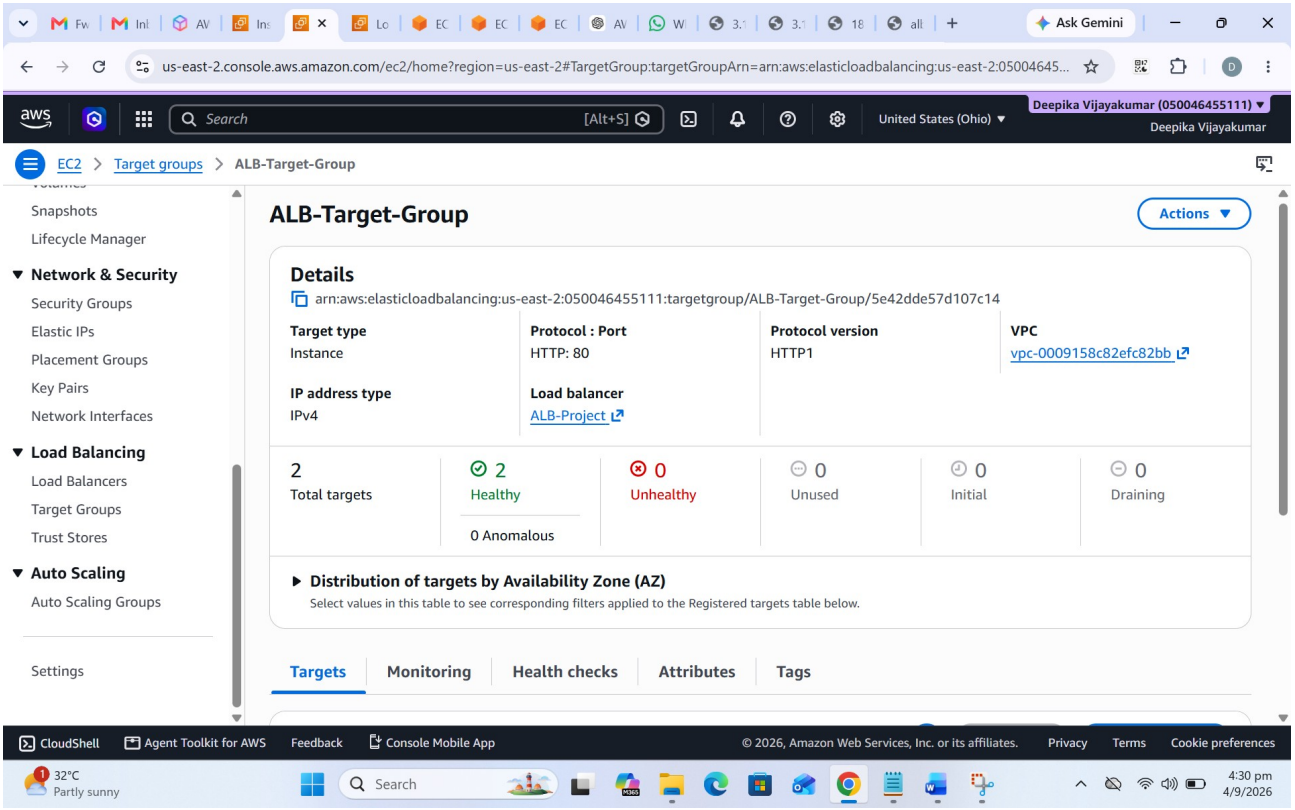


Figure 4 – Target group overview showing two registered targets in the healthy state.

Step 4 – Create the Application Load Balancer

An internet-facing Application Load Balancer named ALB-Project was created in the project VPC. The supplied screenshot shows the ALB in Active status and mapped to two public subnets in Availability Zones us-east-2a and us-east-2b.

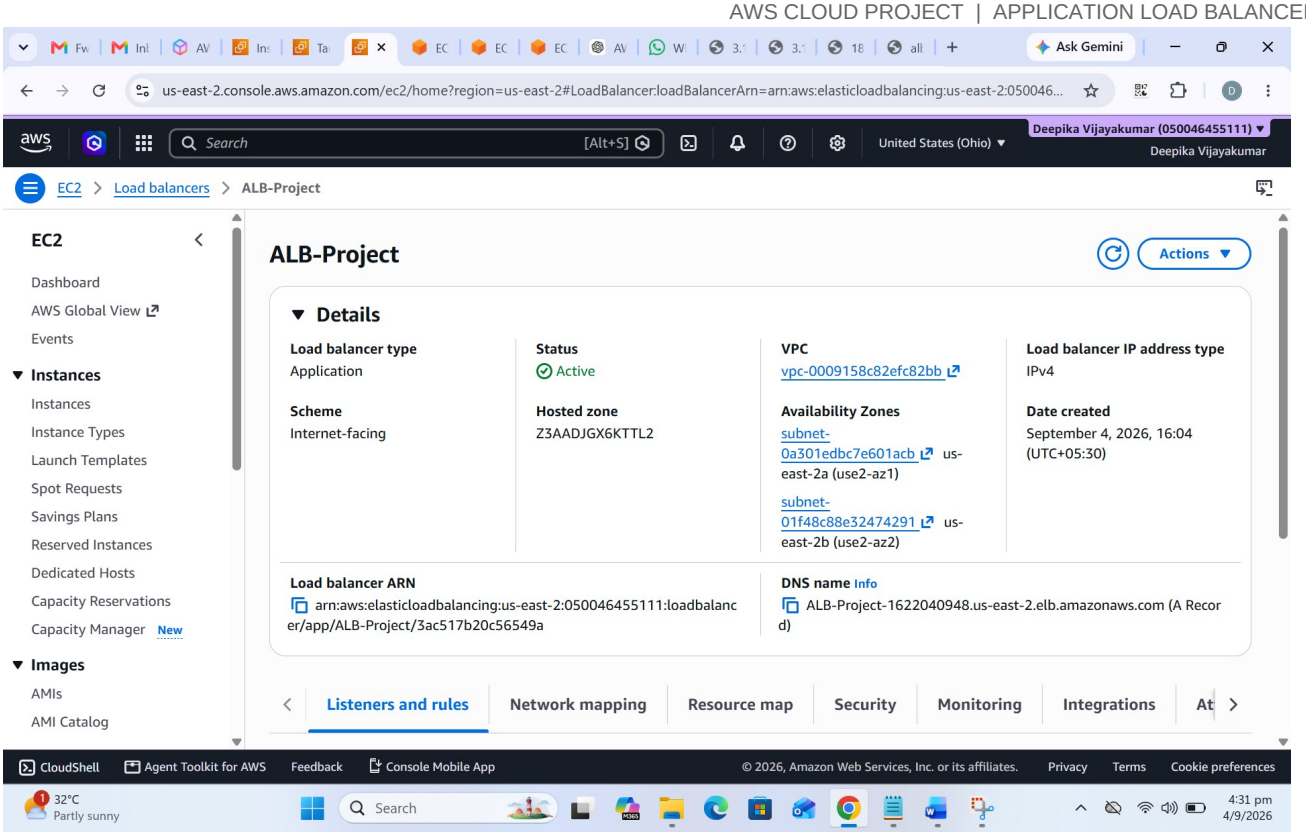


Figure 5 – ALB-Project details, Active status, and Availability Zone mapping.

Step 5 – Verify Registered Targets

The target group registered ALB-Server-1 and ALB-Server-2 on port 80. The screenshot below shows the two registered instance targets and their health status.

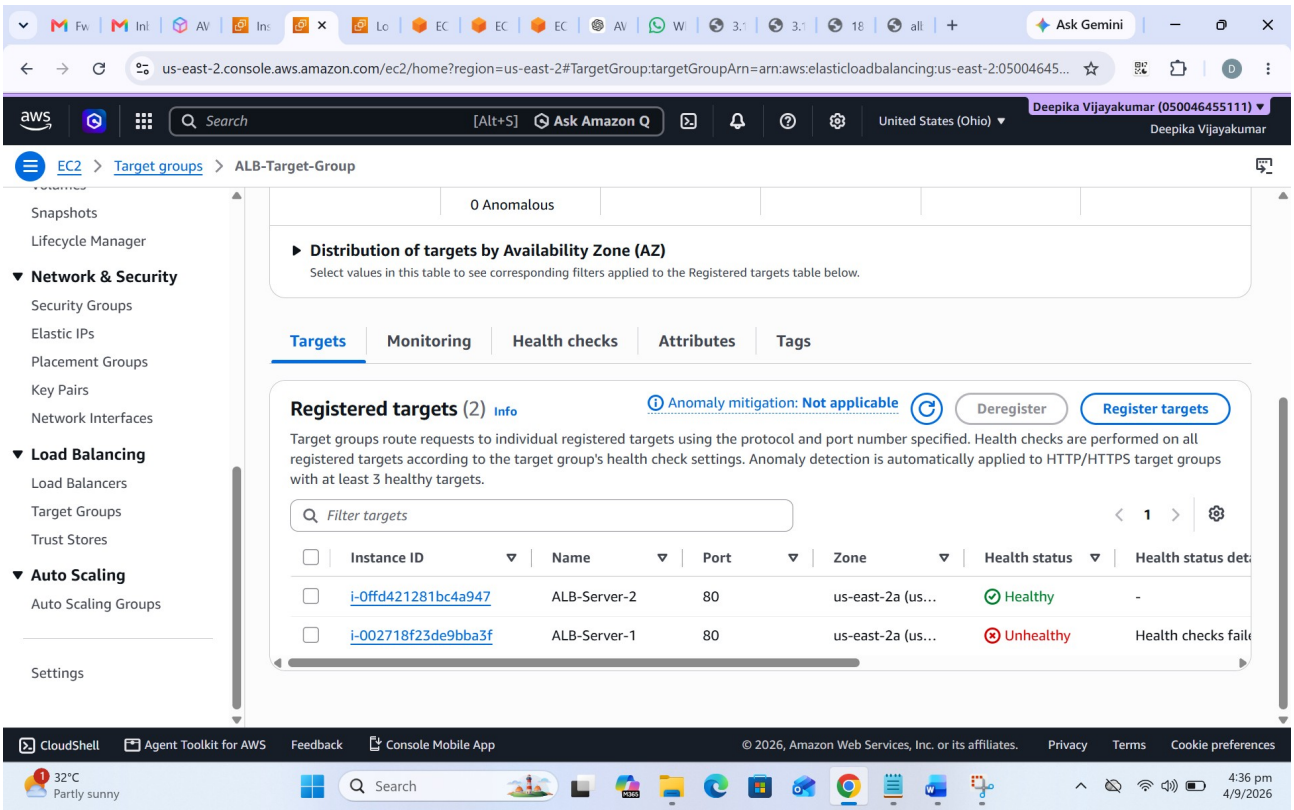


Figure 6 – Registered EC2 targets in ALB-Target-Group (captured during the target-health test).

6. Testing and Validation

Test 1 – Both Targets Healthy

After Apache was running on both instances, the target group reported two healthy targets and zero unhealthy targets.

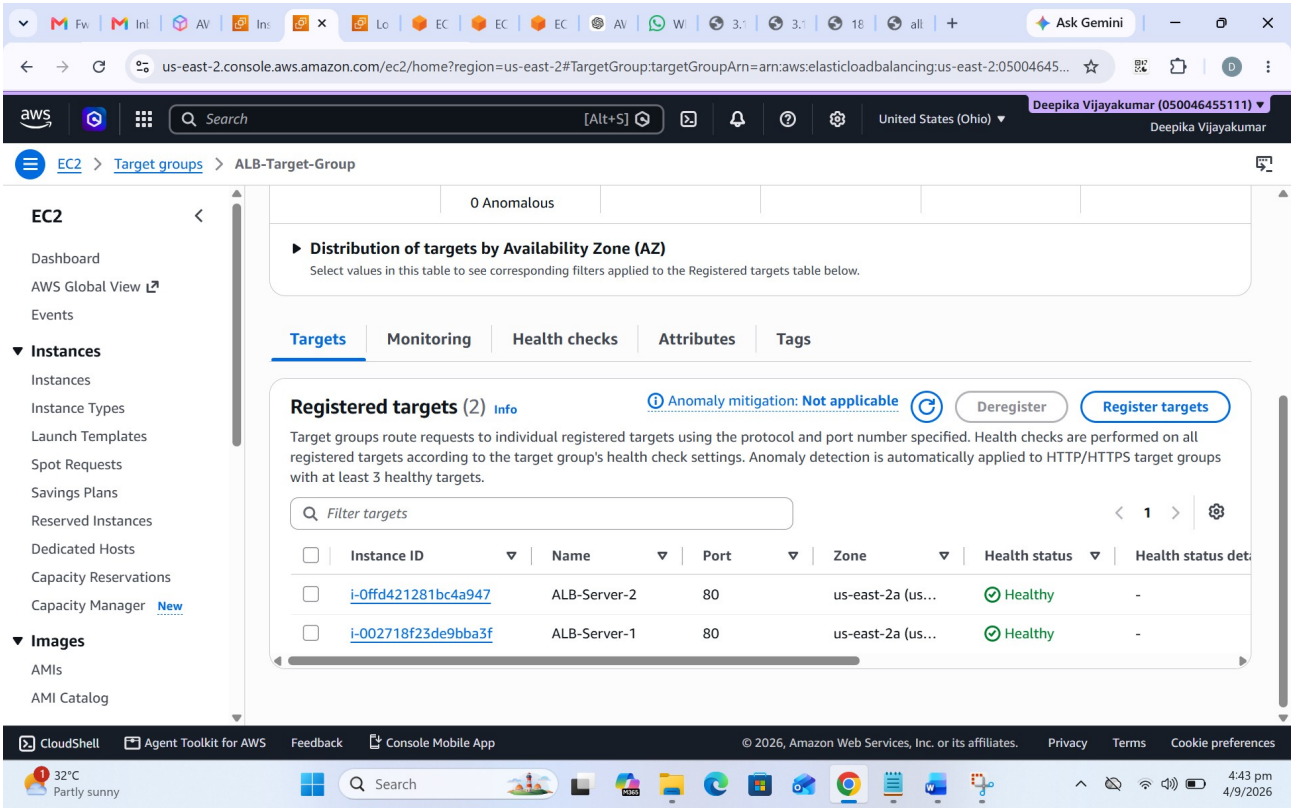


Figure 7 – Both registered targets showing Healthy status.

Test 2 – Simulate Server 1 Failure

Apache was stopped on ALB-Server-1 to simulate an application failure. The service status output shows httpd as inactive (dead).

```
sudo systemctl stop httpd
sudo systemctl status httpd
```

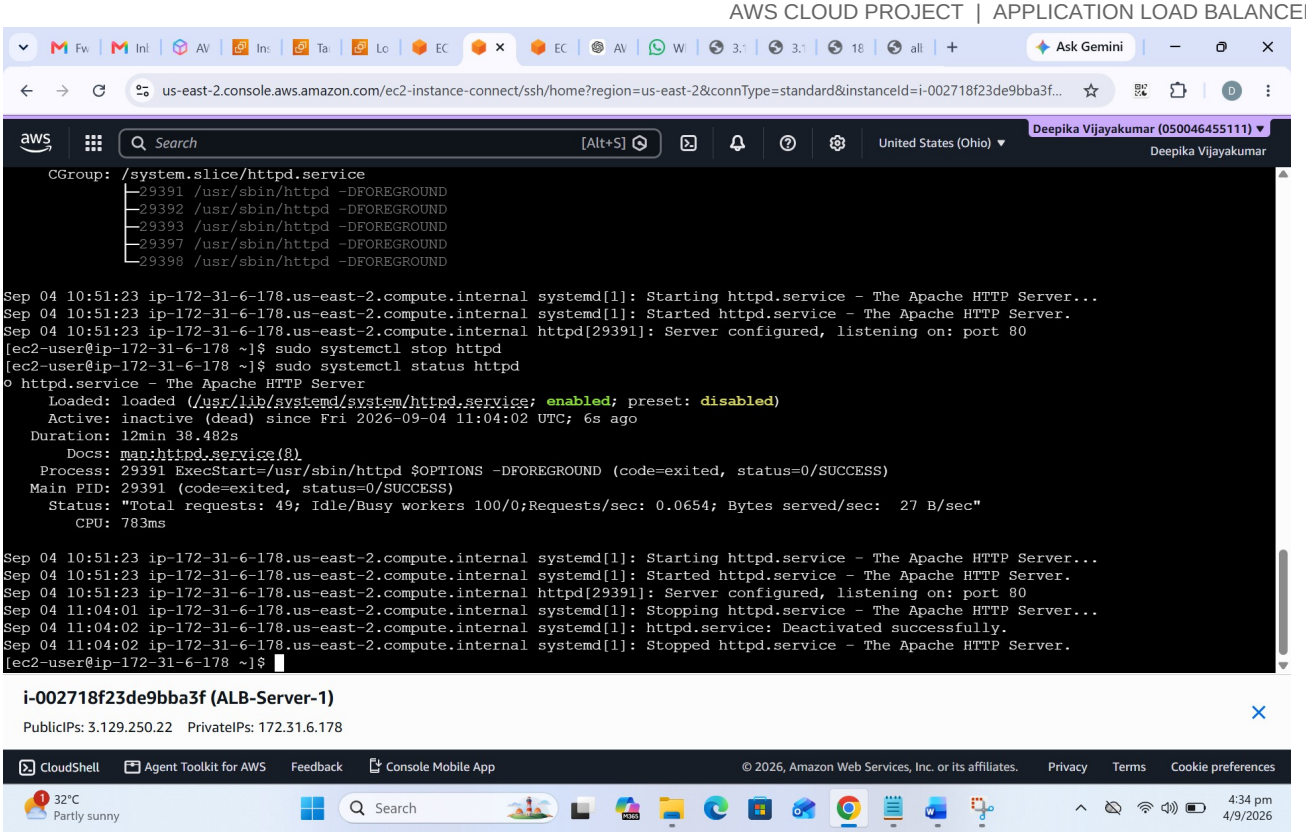


Figure 8 – Apache service stopped on ALB-Server-1 for the failure test.

The target group health check then marked Server 1 unhealthy while Server 2 remained healthy. This is the expected target-health behaviour during the test.

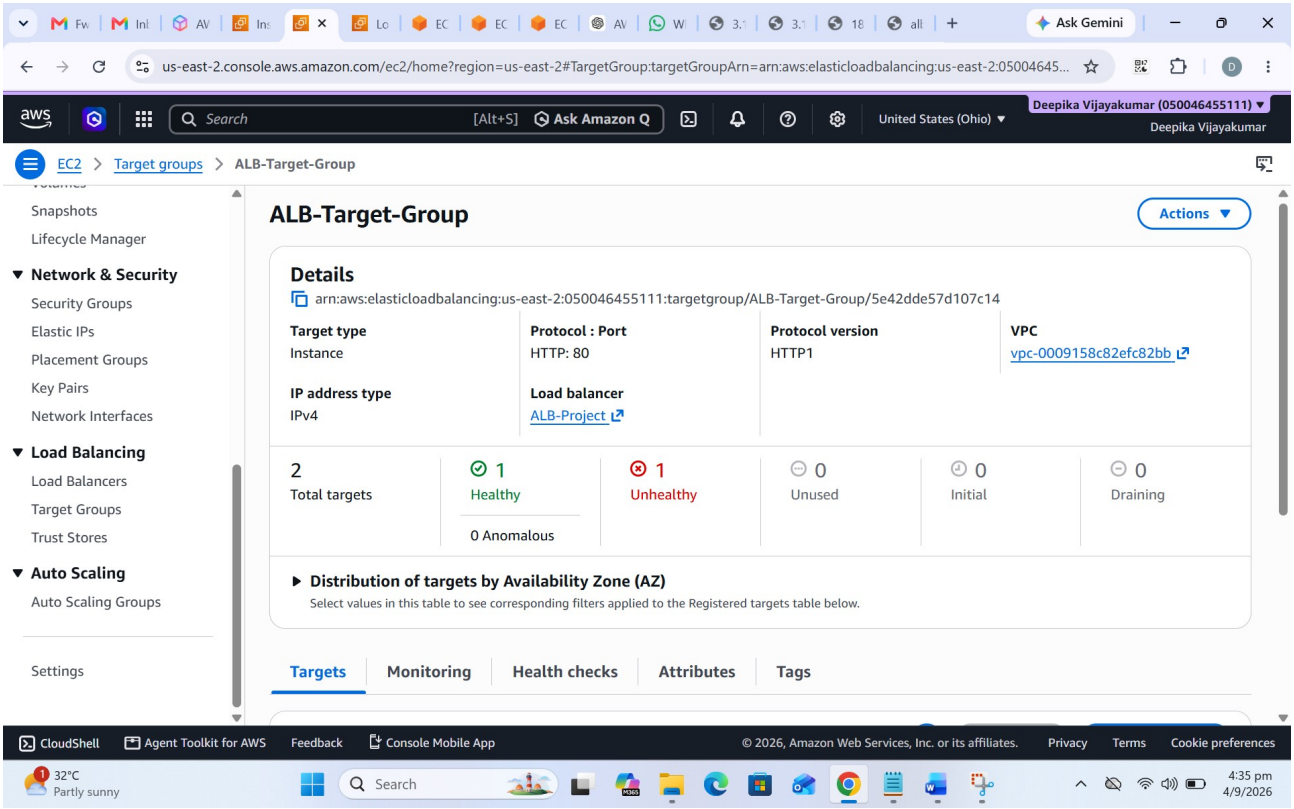


Figure 9 – Target group summary showing one Healthy target and one Unhealthy target.

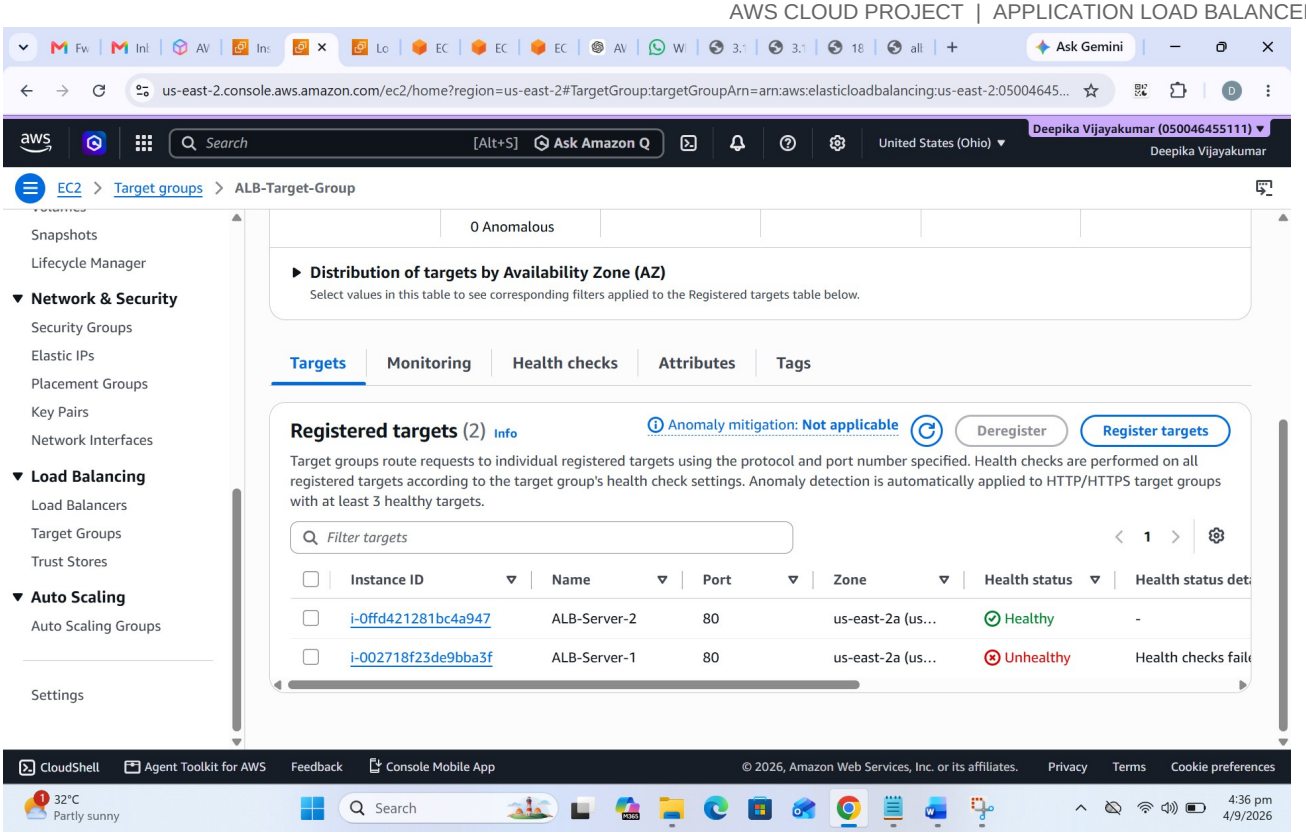


Figure 10 – Registered-target view supplied for the health-check test; status is shown as captured.

Test 3 – Restore Server 1

Apache was started again on ALB-Server-1. The service status output confirms httpd became active (running) again.

```
sudo systemctl start httpd
sudo systemctl enable httpd
sudo systemctl status httpd
```

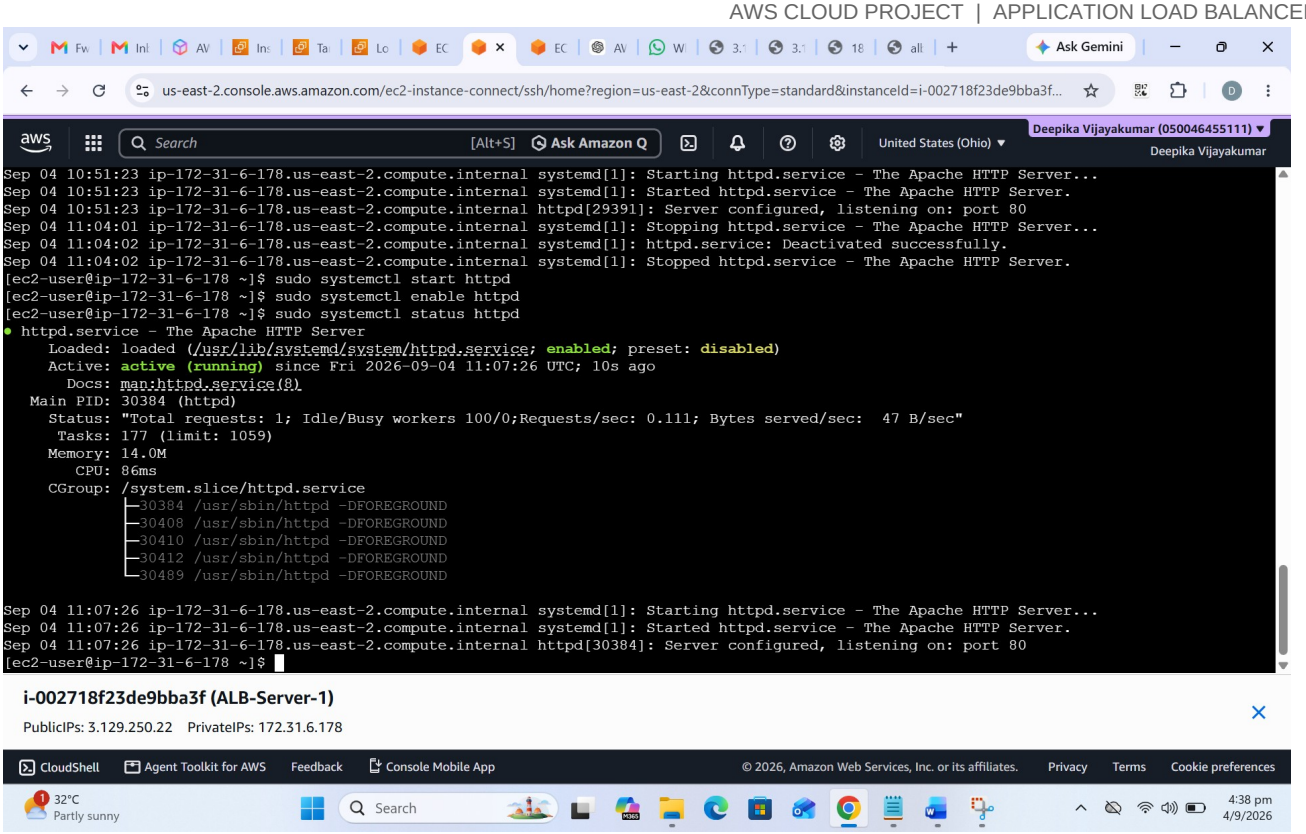


Figure 11 – Apache service active (running) again on ALB-Server-1.

After the health check recovered, both targets were shown as healthy again.

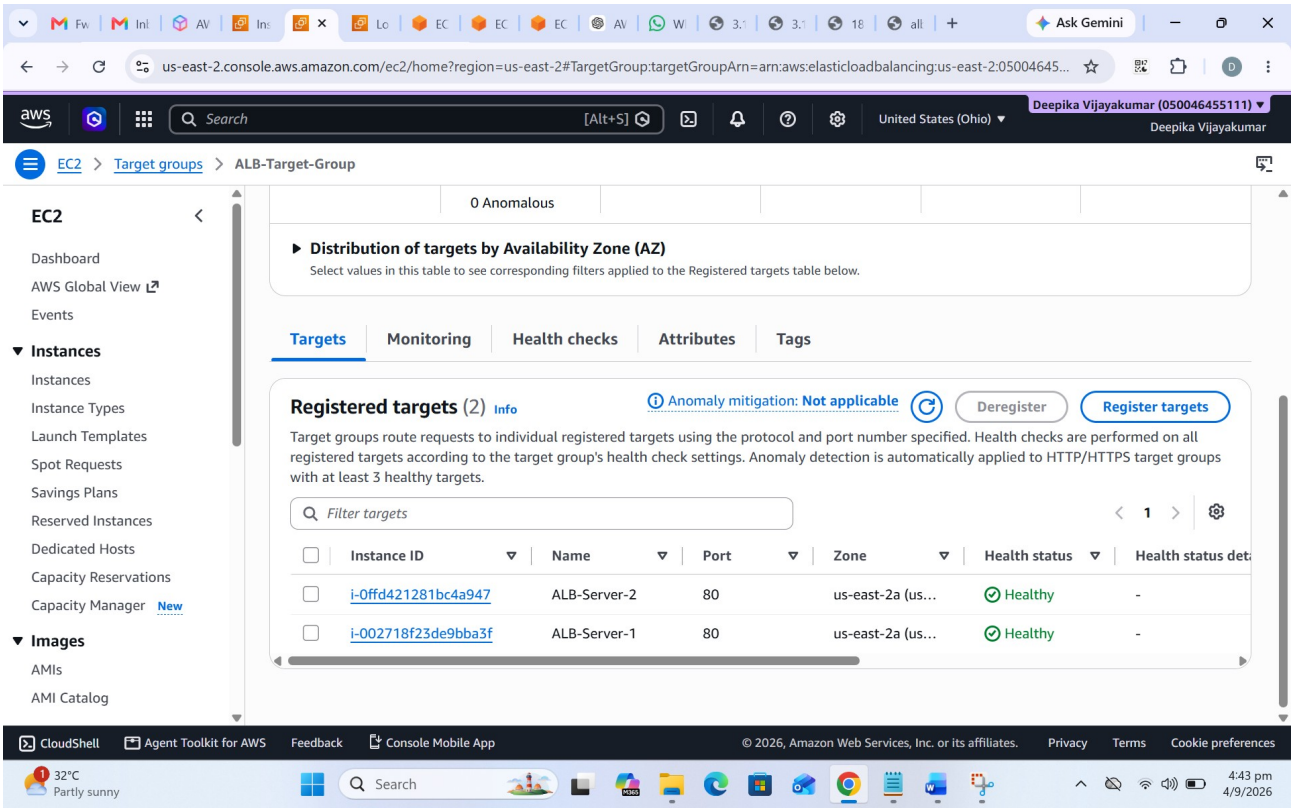


Figure 12 – Target group showing both targets Healthy after recovery.

7. Final Result

- Both EC2 instances ran Apache HTTP Server on port 80.
- The internet-facing ALB was created and associated with the target group.
- The target group health checks identified the stopped Apache service on Server 1 as unhealthy.
- Server 2 remained healthy during the failure test.
- After Apache was restarted on Server 1, the target group showed both targets healthy again.

8. Troubleshooting Checks

- Confirm that the EC2 instance is running.
- Check the Apache service: sudo systemctl status httpd.
- Confirm that Apache is listening on port 80.
- Verify the EC2 security group allows inbound HTTP traffic from the ALB security group.
- Verify the target group protocol and port are HTTP : 80.
- Verify the health check path is /.
- Check that the ALB listener forwards to the correct target group.

9. Key Concepts Learned

Application Load Balancer • Target Groups • Listeners • Health Checks • Healthy and Unhealthy Targets • Traffic Forwarding • High Availability • Failure Detection • EC2 Web Server Management • Apache HTTP Server

10. Project Summary for GitHub

Implemented an AWS high-availability web application using an internet-facing Application Load Balancer and two Amazon Linux 2023 EC2 instances running Apache. Configured an HTTP target group and health checks, verified both targets as healthy, simulated an application failure by stopping Apache on Server 1, observed the unhealthy target state while Server 2 remained healthy, and restored Server 1 to a healthy state after restarting Apache.

11. Scope Note

This evidence documents the Application Load Balancer and EC2 health-check/failover project. It does not show an Auto Scaling Group configuration or scale-out/scale-in test. Add separate Auto Scaling screenshots and configuration details before describing this as an ALB + Auto Scaling project.